PREZENTAREA APLICAȚIEI *ACTIVE MIND EXPLORER* (AME)

Bălălău Andrei-Valentin

Ianuarie 2026

1. Introducere - Prezentare generală, idei și arhitectură

1.1 Introducere | tematică și precizări de funcționalitate

Active Mind Explorer (AME) este un proiect gândit sub forma unui sistem distribuit, al unui program compus din două (sub)aplicații independente care "discută" între ele prin rețea, folosind o arhitectură client-server. Tema propusă se referă la evidența prezenței participării la cursuri online pentru o platfomă ce oferă servicii educaționale, iar din dorința de a oferi o experiență cât mai vastă de utilizare, incluzând aici atât partea funcțională, cea de raționament, de backend (structură baze de date și programare de aplicații în Java, cât și cea creativă, de design (prin frontend și statistici), am considerat că pot îmbina mecanismele comune ale proiectelor din cadrul Aplicațiilor Web cu suport Java și a Bazelor de date într-un singur pachet, în limita timpului disponibil și a respectării cerințelor referitoare la funcționalitatea și constrângerile aplicației.

Fiind o aplicație ce constă în modificarea (inserarea/ actualizarea /ștergerea/ vizualizarea) datelor de la nivelul unei baze de date formate din șapte tabele prin intermediul SQL Server, dar și prin calcularea unor elemente de statistică pentru fiecare meniu / tabel în parte, am vrut să creez o experiență completă de utilizare, să descopăr mai multe funcționalități din arhitectura Spring Boot, grație JPA/ ORM, să automatizez procesul unde era nevoie pentru a mai reduce complexitatea în punctele cheie și de a folosi funcționalitățile SQL prin intermediul cererilor, al interogărilor simple/complexe unde era strict necesar din perspectiva prelucrării datelor, pentru a respecta structura propusă în ambele situații. Mai simplu spus – fiecare proiect se găsește într-o formă în această aplicație. Funcționalitățile Java (mai precis JavaFX) pe partea de UI au fost și ele analizate pentru a realiza o interfață adecvată, prietenoasă, relativ responsive (cel puțin în partea de meniu principal). Sunt și aici câteva funcționalități extra, pe care le voi menționa în părțile următoare.

Această decizie de implementare a fost considerată pentru a separa complet interfața grafică (rulată pe dispozitivul utilizatorului) de logica de procesare și interconectare a datelor și, implicit, a operațiilor de la nivelul bazei de date (aflate pe server) și de a oferi, pe viitor *(de ce nu?)* o scalabilitate bună, o mentenață sau extindere mai rapidă, ușoară a funcționalităților, prin distingerea adecvată a acestor două referințe/ mecanisme, respectiv prin asigurarea unui intermediator (serverul) în relația client, care poate crește gradul de securitate al aplicației (prin evitarea compromiterii directe a datelor relevante).

Inspirația pentru structura bazei de date propuse, și a funcționalităților (evident, redimensionate la un nivel optim pentru volumul de lucru) constă în urma unui curs de 6 săptămâni de familiarizare cu mediul Salesforce, din partea OSF Academy, axat pe

gestionarea și personalizarea bazelor de date cloud pentru mediul de afaceri. Procesul de învățare a combinat studiul individual (prin module, provocări practice și utilizarea ulterioară a limbajelor Apex, SOQL, SOSL și de introducere în API-urile SOAP și REST) cu sesiuni săptămânale de mentorat, progresul fiind validat constant prin exerciții tehnice și implicare activă în discuțiile cu trainerii. Deși în situația mea conta mult rezolvarea de teste/ challenges și module din cadrul Trailhead pentru acumularea de puncte, m-am gândit, în conformitate și cu tema, să extind acest lucru prin contorizarea sesiunilor live cu aceeași pondere alocat pentru partea de progres asincron și să-mi valorific o parte din cunoștințele dobândite în cadrul acestui curs (și nu numai).

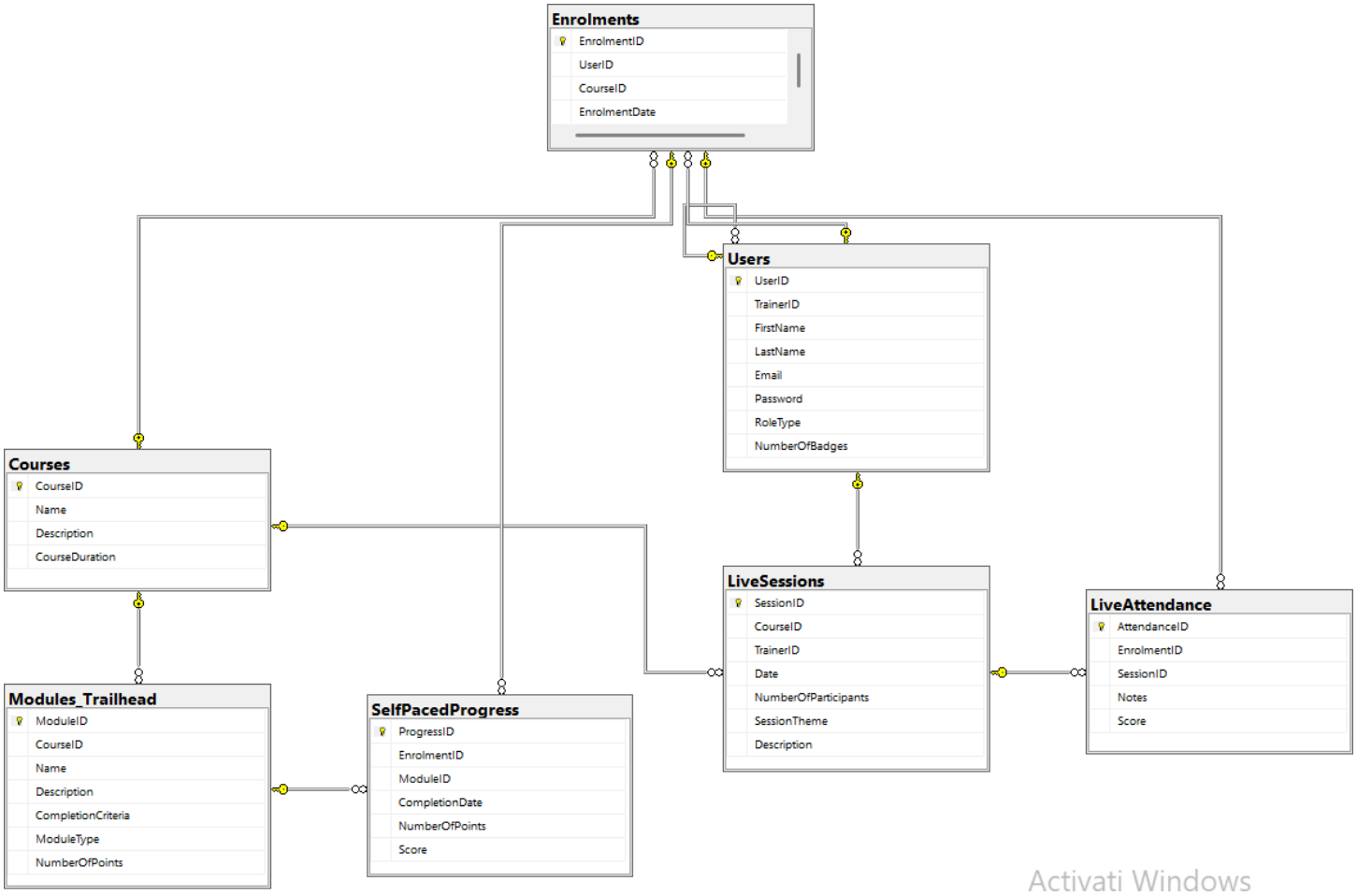
În cadrul acestui document, întrucât conținutul e de ordinea zecilor de fișiere și a sutelor de linii de cod, nu voi putea acoperi detaliat sau în întregime mecanismele folosite la fiecare verificare de date, de gestionare a legăturii SQL – Java prin anumite adnotări sau de descriere a atributelor / elementelor de afișare a interfeței. Desigur, în general multe bucăți de cod replică niște modele pentru fiecare fereastră în materie de funcționalitate sau vizualizare, cu distincțiile lor potrivite opțiunii / secțiunii respective. În continuare, vor fi prezentate ideile din prisma funcționalităților dorite ale aplicației pornind de la detalii tehnice și de organizare și menținere activă a seturilor de date prin intermediul SQL Server, cât implementate prin Java/Spring Boot.

1.2 Definirea bazei de date și a funcționalităților primordiale

Pornind de la reperele OSF Academy, mi-am imaginat o bază de date ce cuprinde mai mult decât date individuale – mă refer în această circumstanță și la aspectele care fac legătura și constituie magia manipulării complexe a informațiilor. Și uneori, pentru a gestiona nu doar faptul că există useri înregistrați, dar și pentru a aplica la cursuri și a lucra la modulele propuse sau participa la sesiunile live propuse, partea de actualizare, adăugare și ștergere devine destul de delicată la început. În tabelul de mai jos este ilustrată replica tabelelor și a funcționării analizate.

Tabel / secțiune din BD	Scop	Câmpuri	Cheie primară (+ externă)	Observații
Utilizatori	Stochează toți participanții: cursanți, traineri, administratori ai platformei, nr_badges_primite	Nume, Prenume, Email, Parolă, tip_rol, ID_trainer	**ID_Utilizator**	Autoreferențiere (Utilizatori <-> Traineri; Trainerii au ID *null*, dar restul cursanților au *ID_Trainer* drept o înregistrare în *Utilizatori* ca *ID_Utilizator*

Cursuri	Detalii despre programul de training, ex: "Salesforce Dev Core", “Salesforce Admin”, etc	Nume, Descriere, Durata	**ID_Curs**	-
Înscrieri	Leagă un cursant la un curs și înregistrează perioada de participare.	Data inscrierii	**ID_Inscrieri** + ID_Curs, ID_Utilizator	Rezolvă problema N:N (mai mulți useri care sunt implicați în mai multe cursuri, fie din postura de coordonatori, fie drept cursanți sau colaboratori)
Module Trailhead	Componentele de învățare pe cont propriu (modulele, provocările, obținerea de badge-uri, quiz-uri)	Nume, Descriere, Criteriu de finalizare, Tip modul (lesson/ project), nr_puncte	**ID_Module +** ID_Curs	Apare situația 1:N (un curs conține mai multe module, dar toate aparțin unui curs predefinit)
Progres_Asincron	Urmărește finalizarea fiecărui Modul_Trailhead și obținerea de badge-uri de către cursant. (Verificarea cunoștințelor).	Data finalizare, Nr_puncte_obtinute, Nota/Punctaj	**ID_Progres** + ID_Inscriere, ID_Modul	Pentru progres, trebuie avut în vedere atât componenta compusă de înscriere (utilizator alocat pt curs), cât și modulul alocat
Sesiune_Live	Stochează detaliile despre sesiunile online săptămânale de discuții cu trainer-ul	Data, Nr participanți, Tema, Descriere	**ID_Sesiune** + ID_Curs, ID_Utilizator (Trainer)	Se rezolvă, astfel, o altă dilemă many-to-many: la un curs pot fi alocați mai mulți traineri în sesiuni video, așa cum și trainerii ce fac meeting-uri pot apărea în mai multe cursuri
Participare_Live	Urmărește engagement-ul/prezența activă a fiecărui cursant la o anumită Sesiune_Live	Remarca, Punctaj	**ID_Participare** + ID_Inscriere, ID_Sesiune	Leaga relații N:N între înscrieri (user – course) și sesiunile live aferente la care au acces



Se observă din tabel și diagramă, de asemenea, relațiile 1:N între *Curs* și *Module* (din prisma progresului), respectiv N:N între *Sesiunile_live* și *cursanți (Useri)*

(1) Deci, care este logica aplicației? → Corelarea funcționalităților de administrare și interogare cu baza de date (legate de utilizatori, cursuri, înscrieri, module Trailhead, sesiuni live și progresul detaliat (atât asincron, cât și de prezență), asigurând totodată aplicarea regulilor de acces bazate pe roluri (Cursant, Trainer, Admin, Colaborator).

(2) Alte aspecte propuse legate de funcționalități:
- Gestionarea informațiilor despre cursanți, traineri, cursuri (programele de training), modulele Trailhead și sesiunile live (online).
- Administrarea structurii cursului: definirea modulelor/badge-urilor Trailhead asociate fiecărui curs și programarea sesiunilor live (săptămânale).
- Evidența progresului cursanților: înregistrarea progresului asincron (finalizarea modulelor Trailhead/obținerea de badge-uri/ scalarea

punctajului obținut în cadrul completării modulelor pe intervalul 0 - 100) și a participării active la sesiunile live.
- Raportarea statistică complexă, inclusiv rata de finalizare a cursurilor (scor obținut), top module și nivelul de engagement (participare la sesiuni live).
- Funcții de administrare (bazate pe modelul *CRUD*): adăugare, modificare și ștergere de date (add, update, delete) pentru toate entitățile (utilizatori, cursuri, module, sesiuni etc).[1]
- Căutări, interogări simple și complexe pe baza unor criterii definite (ex: cursanți cu progres peste un anumit prag sau înscriși la mai multe cursuri, utilizatori în sistem, dar neînscriși la sesiunile live (inclusiv traineri / colaboratori), cursuri la care nu avem participanți, module neatașate unui curs și altele).

(3) Am încercat să țin cont și de o serie de constrângeri legate de utilizarea câmpurilor, pentru a păstra integritatea datelor și a marca un comportament adecvat al proiectului final:

Câmpuri (mod acces)	Tip constrângere	Motiv
Utilizatori.Email	UNIQUE	Două conturi nu pot avea același email.
Utilizatori.Tip_Rol	CHECK (Tip_Rol IN ('Cursant','Trainer','Colab'))	Rolul utilizatorului este limitat la valorile definite.
Utilizatori.Nume, Prenume, Email, Parola	NOT NULL	Aceste câmpuri nu pot fi lăsate necompletate.
Cursuri.Nume, Cursuri.Descriere	NOT NULL	Fiecare curs trebuie să aibă un nume și o descriere.
Module_Trailhead.Tip_Modul	CHECK (Tip_Modul IN ('lesson','project'))	Tipurile de module sunt prestabilite.
Progres_Asincron.Nr_Puncte_Obtinute	CHECK (Nr_Puncte_Obtinute >= 0)	Nu se pot obține puncte negative.
Sesiune_Live.Nr_Participanti	CHECK (Nr_Participanti >= 0)	Numărul participanților trebuie să fie pozitiv.
Participare_Live.Punctaj	CHECK (Punctaj BETWEEN 0 AND 100)	Punctajul acordat trebuie să fie între 0 și 100.

2. Alegerile tehnice, arhitectură și structuri de date

[1] Configurarea utilizatorilor, a cursurilor și a progresului live e realizată de ORM (JPA), iar gestionarea modulelor și a progresului asincron se realizează prin SQL nativ. Legătura între funcționalități e asigurată de tabelul “Enrolments”.

În timpul dezvoltării, am luat câteva decizii specifice legate de tipurile de date și structurile folosite, pentru a evita probleme comune în Java.

2.1 Gestionarea datei calendaristice (prin intermediul LocalDate)

Pentru toate câmpurile care implică date (de exemplu, data finalizării unui modul), am ales să folosesc atât funcționalitatea string-urilor în configurarea formatului de transmitere a datelor, alături de clasa *java.time.LocalDate* din Java 8, în detrimentul vechii clase *java.util.Date*. Date este o clasă mai veche care gestionează dificil fusurile orare și este mutabilă (poate fi modificată accidental), în timp ce *LocalDate* este mai clară și lucrează strict cu data, fără a complica lucrurile cu ora sau timezone-ul, ceea ce este suficient pentru nevoile noastre (data finalizării cursului). Abordarea strategiei hibridă pentru gestionarea datelor se reflectă conform arhitecturii:

- **Pe server:** Folosesc *java.time.LocalDate* pentru a lucra corect cu baza de date SQL Server (tipul DATE).
- **La transport date (JSON):** Datele circulă sub formă de *String* în format *ISO-8601* ("2026-01-01").
- **Pe client (Model):** În unele clase de model (exemplu: *LiveSession*), am ales să păstrez câmpul ca *String date* pentru a evita erorile de deserializare *Jackson* (care ar necesita niște configurații mai complexe).

Pentru a beneficia totuși de puterea LocalDate în logică (la formatarea "dd/MM/yyyy"), am creat metode helper precum *getParsedDate()* care convertesc string-ul în *LocalDate* doar atunci când este nevoie ("Lazy conversion").

2.2 Utilizarea și definirea diferențelor *Integer vs int*

În clasele de model (precum User sau Course), pentru ID-uri am folosit clasa Integer și nu tipul primitiv int (pentru partea de id Trainer). Am făcut acest lucru deoarece în bazele de date relaționale, o cheie străină poate fi uneori NULL (de exemplu, un student care a fost creat dar încă nu a fost repartizat unui trainer). Dacă foloseam int, valoarea implicită ar fi fost 0, ceea ce ar fi putut fi interpretat greșit ca fiind un ID valid. Folosind Integer, pot verifica simplu dacă un câmp este null.

2.3 Șablonul Singleton pentru stabilirea mecanismului de logare (UserSession)

Pentru a ține minte utilizatorul care s-a autentificat, am creat clasa *UserSession* folosind șablonul Singleton, ceea ce înseamnă că o singură instanță a acestei clase există în toată aplicația client. Astfel, se pot accesa datele utilizatorului (nume, rol, ID) din orice fereastră (*Dashboard, Users, Courses, Modules, Progress*) fără a fi nevoie să transmitem obiectul *User* ca parametru de la o metodă la alta, din fereastră în fereastră, fiind o soluție practică pentru a menține activă starea globală a aplicației client.

Astfel, într-un mod simplificat, aveam nevoie de un centralizator în care să depozitez datele critice, accesibil oricând. Aici, metoda *UserSession.getInstance()* este punctul de acces global. Orice controller o poate apela pentru a afla cine este logat într-un anumit moment.

Datele utilizatorului: Reține obiectul "complet " *modifiedUser* (ID, nume, rol). Astfel, interfața poate decide instantaneu ce butoane să ascundă (drepturi de acces), verificând doar *getInstance().getLoggedInUser().getRole().* De exemplu, colaboratorii au acces exclusiv la conținutul din Progress Panel (vizualizarea/editarea componentelor destinate liveSession, liveAttendance, Enrolment și selfPacedProgress.

Preferințele globale: Stochează starea *isDarkMode*. Dacă utilizatorul schimbă tema într-un ecran, Singleton-ul reține această opțiune, iar celelalte ecrane o pot citi și aplica automat, direct.

2.4 Arhitectura distribuită | Model – view – controller – (service): schematizare și exemple

Așa cumam evidențiat, pe seama informațiilor succinte anterioare, ambele părți ale aplicației respectă intern șablonul **MVC**, dar la niveluri diferite:

1. **Nivelul Client (Frontend – JavaFX și preluare date de la server):**
   - ✓ Rulează pe dispozitivul utilizatorului, iar fișierele se găsesc în src\main\javasrc\main\java\com\osf\coursemanagement
   - ✓ MVC-ul pt client se regăsește sub forma:
     - *View:* Fișierele .fxml (interfața grafică - *splash-view.fxml, module-management-view.fxml* - similare la nivel de structură cu standardul html)
     - *Controller:* Clasele Java care răspund la butoane (ex: DashboardController, LoginController, UserManagementController, etc și care pot îngloba atât părți de funcționalitate, cât și de grafică și stilizare (exemplu: progress bar).

- *Model:* Clasele Java simple (User, Course, Enrolment, LiveAttendance, etc) care țin datele în memorie.

2. **Nivelul Server (Backend - Spring Boot):**

   ✓ Rulează independent, poate fi pe alt calculator (Distributed Server).

   ✓ MVC-ul serverului cuprinde din perspectiva:

   - *Model (exemplu: AuthService.java):* se reflectă prin Service Layer - fiind mecanismul central de decizie și baza de date (Repository, Interfețe care extind JpaRepository). Service conține logica de *business* (verifică parola, calculează statistici, verifică unicitatea email-ului, s.a.m.d). Folosește adnotarea @*Autowired* pentru a *injecta/marca* repository-urile de care are nevoie. În schimb, @Repository gestionează scrierea și citirea din baza de date SQL Server. Nu a fost nevoie, așadar, să scriu cod SQL manual pentru operațiile simple (CRUD) din cadrul User Management și Course Management, Spring generându-le automat.
   - *View:* Răspunsul JSON trimis înapoi clientului, analog cu nivelul client.
   - *Controller (exemplu AuthController.java):* expune punctele de acces (endpoints), primind cerere HTTP (ex: POST /login) folosind adnotarea @RestController; extrage apoi datele din JSON (@RequestBody) și le trimite mai departe.

Un aspect pe care am uitat să-l menționez se referă la faptul că o astfel de abordare este, de asemenea, mai flexibilă decât o aplicație monolitică deoarece permite, teoretic, ca pe viitor să pot integra conecta și o interfață web sau o aplicație mobilă la același server, fără a modifica partea de backend.

2.5 Paralele cu ecosistemul Salesforce

Logica de Business este similară modului în care clasele Apex gestionează logica pe server în Salesforce, în această aplicație am utilizat clase de Service în Spring Boot (ModuleService, AuthService) pentru a încapsula regulile de business. Operațiile DML din Salesforce (insert, update, delete) au echivalent direct în metodele save() și delete() din repository-urile Spring Data JPA.

Pentru interogările complexe (ex: calculul insignelor), am utilizat SQL nativ, similar modului în care SOQL (Salesforce Object Query Language) este folosit pentru a extrage date specifice relaționate. La fel ca în Salesforce, unde putem expune clase

Apex ca servicii REST pentru integrări externe, backend-ul Spring Boot expune endpoints REST (@RestController) care servesc date în format JSON.

3. Implementarea Backend-ului (Spring Boot & MVC)

3.1 Configurare și generare proiect. Adnotări

Proiectul a fost generat utilizând Spring Initializr, incluzând dependențele cheie: Spring Web (pentru REST API), Spring Data JPA (pentru acces baza de date) și SQL Server Driver. Configurarea accesului la baza de date este centralizată în fișierul *application.properties*, unde am definit:

- URL-ul de conectare (*jdbc:sqlserver://...)*
- Strategia de generare a tabelelor (*ddl-auto=update*), care permite *Hibernate* să actualizeze automat schema bazei de date fără a șterge datele existente.

Pentru a lega componentele din arhitectura MVC ("wiring"), am folosit adnotările standard Spring, ce includ:

- *@SpringBootApplication*: pentru clasa principală care pornește serverul.
- *@Entity* și *@Table*: pe clasele de model (ex: User), pentru a spune Hibernate-ului că acea clasă corespunde unei tabele în baza de date.
- *@Autowired*: Pentru *Dependency Injection* (DI). Spring "forțează" automat instanța de UserRepository în AuthService, fără să folosim *new UserRepository()*.
- *@RequestMapping*, *@GetMapping*, *@PostMapping*: Pentru a realiza map-urile URL-urilor web la metodele Java.

3.2 Compunerea sistemelor complexe prin Service

Un aspect important al implementării backend este interacțiunea dintre servicii. Nu m-am limitat la operații simple de tip CRUD (Create-Read-Update-Delete), ci am încercat formarea unor fluxuri complexe de business, aflate în legătură, folosind mai multe servicii care comunică între ele.

Primul exemplu care se pliază pe această idee este ștergerea unui utilizator (*AuthService.deleteUser*). În mod obișnuit, recomandat, nu putem șterge pur și simplu un utilizator din baza de date, făcând abstracție de restul dependențelor cu acea înregistrare deoarece ar rămâne date orfane (note, prezențe), irelevante. De aceea, am implementat o logică de ștergere "în cascadă" manuală, care demonstrează eficacitatea mecanismului, în varianta sa limitată de bază prin următoarele etape:

1. AuthService primește comanda de ștergere
2. În loc să șteargă direct, el apelează un alt serviciu: *EnrolmentService*, ce se ocupă de gestionarea componentei de date din înscrerile la curs
3. EnrolmentService execută logica grea, în sensul că:
   - Șterge mai întâi progresul asincron (*SelfPacedProgress*).
   - Actualizează sesiunile live (*LiveSession*), scăzând numărul de participanți cu 1, pentru a menține consistența datelor.
   - Șterge prezența (*LiveAttendance*).
   - La final (apoi), șterge înscrierea (*Enrolment*).
4. Abia după ce *EnrolmentService* a eliminat restul *conexiunilor,* AuthService șterge utilizatorul definitiv.

Această abordare modulară (un serviciu care folosește alt serviciu prin @Autowired asigură că regulile de business sunt respectate cu strictețe și datele rămân în cele din urmă integre.

4. Funcționalitățile SQL. Interogări

Așa cum am evidențiat anterior, proiectul implică și interogări / utilizare a funcțiilor de tip CRUD prin SQL pentru operații complexe care depășesc capacitățile standard ale ORM-ului, în gestionarea modulelor și a progresului asincron, cât și de obținere a unor date cheie în privința calculării anumitor rezultate, precum numărul de insigne obținut (raportat la gradul de progres pentru SuperBadges), cât și a progresului general (sincron + asincron). Pentru a asigura funcționalitatea adecvată, fiecare *query* este marcat de adnotarea omonimă *(@Query*) și este “atașat” unei metode cu denumire sugestivă (ex: *findTotalScoreByUserIdAndCourseId(int userId, int courseId)*, apelată ulterior în gestionarea logicii aplicației).

Gestionarea comunicării cu baza de date, precum și preluarea datelor necesare aplicației se realizează în componentele adnotate cu *@Repository*. Pe lângă operațiile clasice de INSERT, UPDATE, DELETE aferente celor două tabele, am inclus și alte tipuri de interogări, fiecare dintre acestea fiind detaliate prin exemple și utilizări în secțiunile următoare:

- Interogări simple, cu funcții agregate - SELECT de bază cu COUNT, AVG, SUM
- Interogări simple cu JOIN – pentru combinarea de date din multiple tabele
- Subcereri (interogări complexe) - Interogări imbricate pentru filtrare avansată (EXISTS, IN, NOT IN, scalar subqueries)

- Interogări cu statement-uri speciale - condiționale (CASE WHEN)
- Interogări hibride - Combinație de JOIN-uri, subcereri, agregate pentru analize avansate

4.1 Operații CRUD

În această subecțiune voi pune în evidență operațiile CRUD pentru tabelul de module ale cursurilor (*ModuleRepository.java*), celelalte funcționalități CRUD cu SQL nativ din SelfPacedProgress fiind aproximativ similar (evident, conforme cu structura tabelului).

- Inserarea de valori / înregistrări[2]

```
INSERT INTO Modules_Trailhead
(CourseID, Name, Description, CompletionCriteria, ModuleType, NumberOfPoints)
VALUES
(:courseId, :name, :description, :completionCriteria, :moduleType,
:numberOfPoints)
```

- Actualizarea de valori / înregistrări

```
UPDATE Modules_Trailhead
SET
CourseID = :courseId, Name = :name, Description = :description,
CompletionCriteria = :completionCriteria, ModuleType = :moduleType,
NumberOfPoints = :numberOfPoints
WHERE
ModuleID = :moduleId
```

- Ștergerea de valori / înregistrări

```
DELETE FROM Modules_Trailhead
WHERE ModuleID = :moduleId
```

Vizual, aceste informații pot fi editate / vizualizate prin apăsarea unuia din butoanele corespondente CRUD din Module Management (similar pentru restul tabelelor) și se va deschide un “formular” (dialog window) cu toate câmpurile. În cazul ștergerii și actualizării mai întâi trebuie selectat modulul din listview, altfel va apărea o fereastră de avertizare în acest sens. În plus, la ștergerea înregistrării, utilizatorul e avertizat că acțiunea nu mai poate fi ulterior revocată. Un modul șters/actualizat va determina și eliminarea/actualizarea acestuia din lista de module asociate cursului (sau mutarea lui la un alt curs, dacă a fost selectată această opțiune) și a eliminării (în cazul DELETE) a punctajelor obținute de utilizatorii participanți, păstrând integritatea datelor aflate în corelare.

[2] courseId - ID-ul cursului părinte, name - Numele modulului, description - descrierea modulului, completionCriteria - criteriile de completare/ evaluare, moduleType - Lesson/ProjectBadge/Superbadge, numberOfPoints - punctajul maxim al modulului

Așadar, ștergerea și actualizarea datelor se propagă în cascadă în raport cu *SelfPacedProgress* și User (UserRepository). De exemplu:

```
DELETE FROM SelfPacedProgress
WHERE ModuleID = :moduleId
```

Scopul → ștergerea în cascadă a progresului când se șterge un modul (backend trigger)

```
UPDATE Users
SET NumberOfBadges = :count
WHERE UserID = :userId
```

Scopul → actualizarea numărului total de badge-uri pentru un utilizator. Vizual, după salvarea progresului, contorul de badge-uri se actualizează automat în lista de utilizatori.

4.2 Interogări simple / cu funcții agregate

- COUNT – folosit pentru a determina numărul de înscrieri la curs (EnrolmentRepository.java) / numărul de module/ sesiuni per curs (ModuleRepository.java/ LiveSessionRepository.java), șamd.

```
SELECT COUNT(*)
FROM Enrolments
WHERE CourseID = :courseId
```

În cazul de față, din interfața "Course Management", în panoul *detalii curs*, se observă partea de statistică.

- AVG - media (finală) a punctajelor (în CourseRepository.java)

```
SELECT AVG(Score)
FROM SelfPacedProgress
```

Calculează media tuturor scorurilor din progresul asincron (adică a punctelor scalate prin completarea modulelor). În aplicație se observă pentru utilizator în parte, în coloana tabelului cu progres asincron.

- SELECT simplu - preia toate înscierile pentru un curs, fiind returnat un *List<Enrolment>*, folosit pentru procesare internă (se găsește în *EnrolmentRepository.java*)

```
SELECT *
FROM Enrolments
WHERE CourseID = :courseId
```

4.3 Interogări cu *JOIN*

- JOIN simplu - utilizatori înscriși la un curs (exemplul de mai jos - UserRepository.java), cursurile la care este înscris un utilizator (CourseRepository.java)

```
SELECT u.*
FROM Users u
JOIN Enrolments e ON u.UserID = e.UserID
WHERE e.CourseID = :courseId
```

- JOIN + DISTINCT - găsește cursurile care conțin cel puțin un modul de un anumit tip, eliminând potențiale duplicate (CourseRepository.java)

```
SELECT DISTINCT c.*
FROM Courses c
JOIN Modules_Trailhead m ON c.CourseID = m.CourseID
WHERE m.ModuleType = :moduleType
```

- JOIN + AVG - calculează media scorurilor obținute de un utilizator în sesiunile live (LiveAttendanceRepository.java) - în aplicație se poate vizualiza din User Management → panou detalii utilizator → statistică (tabel) Live Score.

```
SELECT AVG(la.Score)
FROM LiveAttendance la
JOIN Enrolments e ON la.EnrolmentID = e.EnrolmentID
WHERE e.UserID = :userId
```

- JOIN + SUM – pentru calculul scorului total live pentru un utilizator la un curs (exemplul de mai jos- LiveAttendanceRepository.java)/ calculul scorurilor prin rezolvarea modulelor (asincron) ale unui utilizator pentru un curs (SelfPacedProgressRepository.java)

```
SELECT SUM(la.Score)
FROM LiveAttendance la
JOIN Enrolments e ON la.EnrolmentID = e.EnrolmentID
WHERE e.UserID = :userId AND e.CourseID = :courseId
```

- JOIN multiplu - preia datele de prezență (comune) pentru o sesiune live incluzând numele studenților (prin legătura LiveAttendance → Enrolments → Users). În aplicație se observă aceste date selectate din Live Attendance → selectare sesiune → tabel cu participanți (coloanele: First Name, Last Name, Score, Notes și checkbox-ul cu prezența / absența). Utilizatorii nebifați au automat 0 puncte la acea sesiune, care se adaugă la calculul punctajului total.

```
SELECT
la.EnrolmentID as enrolmentId,
la.AttendanceID as attendanceId,
u.FirstName as firstName,
u.LastName  as  lastName,
la.Score as score,
la.Notes  as  notes
FROM LiveAttendance la
```

```
JOIN Enrolments e ON la.EnrolmentID = e.EnrolmentID
JOIN Users u ON e.UserID = u.UserID
WHERE la.SessionID = :sessionId
```

4.4 Interogări complexe

- Subcerere EXISTS - găsește toți utilizatorii (dacă EXISTĂ înregistrări în acest sens) care au participat la cel puțin o sesiune live (UserRepo)
  Reflectare vizuală: Dashboard → Statistici module → "Active Participants" (statistici afișate în alert window)

```
SELECT u.*
FROM Users u
WHERE EXISTS (SELECT 1 FROM Enrolments e WHERE e.UserID = u.UserID
AND EXISTS (SELECT 1 FROM LiveAttendance la WHERE la.EnrolmentID =
e.EnrolmentID))
```

- Subcerere cu COUNT – identifică utilizatorii care nu sunt înscriși la niciun curs, fiind un scalar subquery (adică returnează o valoare unică). Utilizatorii neînregistrați apar în *Stats.*

```
SELECT *
FROM Users u
WHERE (SELECT COUNT(*) FROM Enrolments e WHERE e.UserID = u.UserID) = 0
```

- Subcerere cu IN + GROUP BY + HAVING - găsește utilizatorii cu mai mult de X înscrieri (implicit, setat la 1). Acești useri apar, de asemenea, în User Stats.

```
SELECT *
FROM Users u
WHERE u.UserID IN (SELECT e.UserID FROM Enrolments e GROUP BY e.UserID HAVING
COUNT(e.EnrolmentID) > :minEnrolments)
```

- Subcerere NOT IN - cursurile la care nu s-a înscris nimeni (CourseRepo - de văzut din Course Stats → empty courses)

```
SELECT *
FROM Courses c
WHERE c.CourseID NOT IN (SELECT e.CourseID FROM Enrolments e)
```

- Subcerere Corelată cu COUNT – pentru cursuri cu multe module (CourseRepo), sau pentru module neutilizate cu NOT EXISTS (ModuleRepo), etc

4.5 Interogări hibride (JOIN + subcereri + CASE + agregate)

- Utilizatori cu scor peste media cursului la care sunt înscriși. Prin JOIN-uri se conectează Users → Enrolments → SelfPacedProgress, iar calculul mediei pentru cursul specific (e.CourseID) se face prin subcereri. În aplicație, se pot vedea din User Stats participanții cu scor mai mare decât media.

```
SELECT DISTINCT u.*
FROM Users u
JOIN Enrolments e ON u.UserID = e.UserID
JOIN SelfPacedProgress sp ON e.EnrolmentID = sp.EnrolmentID
WHERE sp.Score > (SELECT AVG(sp2.Score) FROM SelfPacedProgress sp2 JOIN
Enrolments e2 ON sp2.EnrolmentID = e2.EnrolmentID WHERE e2.CourseID =
e.CourseID)
```

- Calcul Badge-urilor cu JOIN Multiplu și CASE (SelfPacedProgressRepo)

```
SELECT SUM(CASE WHEN TRIM(UPPER(m.ModuleType)) = 'SUPERBADGE' THEN (CASE WHEN
sp.Score >= 50 THEN 3 WHEN sp.Score > 0 THEN 1 ELSE 0 END) ELSE (CASE WHEN
sp.Score > 0 THEN 1
ELSE 0 END)END)
FROM SelfPacedProgress sp
JOIN Enrolments e ON sp.EnrolmentID = e.EnrolmentID
JOIN Modules_Trailhead m ON sp.ModuleID = m.ModuleID
WHERE e.UserID = :userId AND e.CourseID = :courseId
```

Calculează numărul total de badge-uri pentru un utilizator (@Param userId - ID-ul utilizatorului). Superbadge-ul reprezintă efectiv 3 badge-uri dacă scorul ≥ 50% sau 1 badge dacă scor > 0% și sub 50%. Mecanismul pentru badge-ul clasic e de un badge dacă scorul > 0%. În aplicație se actualizează automat în funcție de datele introduse/editate/șterse, fiind observabil în Badges Progress → selectare user → Header "Total Badges: X" (actualizat automat la salvare) sau din User Panel → selectare user și verificare nr badges din tabelul cu rezultate.

4.6 Interogări cu câmp/ parametru variabil

Toate interogările din aplicație folosesc parametri (@Param) pentru a preveni SQL injection și a permite reutilizare. Multe din exemplele ilustrate anterior se bazează pe acești parametri. În tabelul de mai jos se poate observa varietatea lor:

Tip parametru	Exemplu	Utilizare
ID unic	:userId, :courseId	Căutare după identificatori primari
String	:moduleType, :name	Filtrare pe text
Numeric	:minEnrolments, :score	Comparații numerice
Date	:completionDate	Filtrare temporală

Interogările SQL folosesc parametri variabili pentru a urmări activitatea utilizatorilor în timp real, incluzând identificare (nume, ID), date temporale (completion dates), și reflectarea vizuală în interfață. Aplicația folosește aceste tipuri de parametri pentru a înregistra și afișa activitatea utilizatorilor, de exemplu:

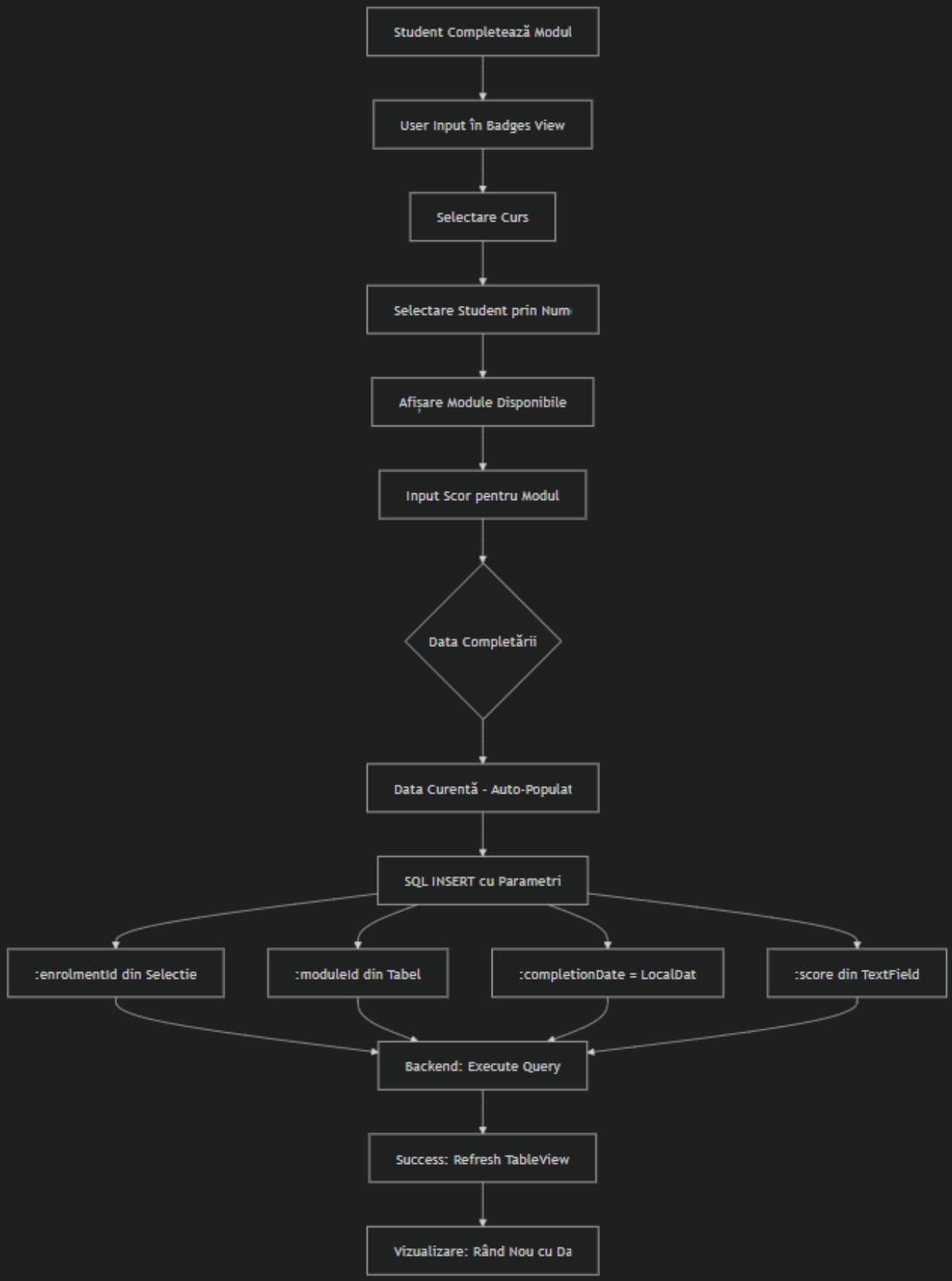
```
INSERT INTO SelfPacedProgress
(EnrolmentID, ModuleID, CompletionDate, NumberOfPoints, Score)
VALUES
(:enrolmentId, :moduleId, :completionDate, :numberOfPoints, :score)
```

(Înregistrarea progresului asincron cu date temporale - LocalDate, din java.time.LocalDate - format SQL: *YYYY-MM-DD*)
În aplicație, se poate observa efectul din Badge Progress Management
Componente:

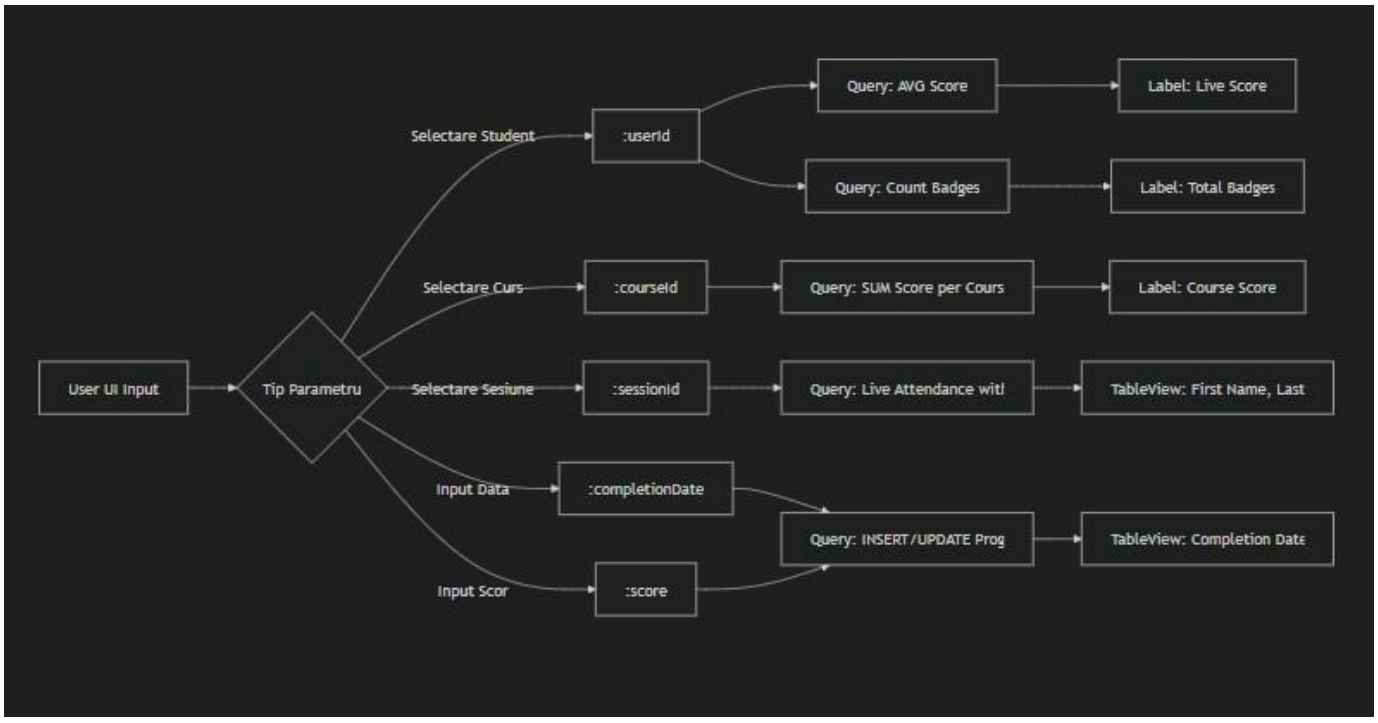
- ComboBox pentru selectare curs
- ComboBox pentru selectare student (populat cu firstName + lastName)
- TableView cu coloane:
  - Module Name (String)
  - Completion Date (LocalDate) - afișare dată
  - Score (Integer - editabil)
  - Max Points (Integer)

Coloana "Completion Date" afișează formatul dd MMM yyyy (ex: "23 Ian 2026")

Alte exemple mai includ prezența live la sesiuni cu nume participanți și parametrul aferent SessionID sau actualizarea progresului cu modificare de dată, unde user-ul face double-click pe celula cu data în TableView → apare *DatePicker* pentru modificare; la schimbare → auto-save → query UPDATE cu noul :completionDate, iar TableView se reîmprospătează cu noua dată. A se urmări diagramele de mai jos pentru ilustrări detaliate ale acestor mecanisme.


Student Completează Modul
User Input în Badges View
Selectare Curs
Selectare Student prin Num
Afișare Module Disponibile
Input Scor pentru Modul
Data Completării
Data Curentă - Auto-Populat
SQL INSERT cu Parametri
:enrolmentId din Selectie
:moduleId din Tabel
:completionDate = LocalDat
:score din TextField
Backend: Execute Query
Success: Refresh TableView
Vizualizare: Rând Nou cu Da

Trainer Accesează Live Atte
Selectare Curs din ComboBo
Backend: Load LiveSessions
Afișare Lista Sesiuni cu Date
Trainer Selectează Sesiune
Parametru: sessionId
Query JOIN Multiplu
Extrage FirstName + LastNa
Populare TableView
Coloană: First Name
Coloană: Last Name
Coloană: Score Editable
Coloană: Notes Editable


User UI Input
Tip Parametru
Selectare Student
:userId
Query: AVG Score
Label: Live Score
Query: Count Badges
Label: Total Badges
Selectare Curs
:courseId
Query: SUM Score per Cours
Label: Course Score
Selectare Sesiune
:sessionId
Query: Live Attendance wit
TableView: First Name, Last
Input Data
:completionDate
Query: INSERT/UPDATE Prog
TableView: Completion Date
Input Scor
:score

Aplicația folosește parametri variabili pentru:

- ✓ Identificare utilizatori: prin *:userId* pentru calcule personalizate
- ✓ Tracking temporal: prin :completionDate (LocalDate) pentru istoricul activității
- ✓ Afișare nume: JOIN-uri cu Users pentru FirstName și LastName
- ✓ Filtrare contextuală: Combinație :userId + :courseId pentru rapoarte detaliate
- ✓ Vizualizare în timp real: update automat al UI la modificarea parametrilor

Fiecare parametru este mapat direct la o componentă UI vizibilă, oferind feedback instant utilizatorului despre activitatea sa (și nu numai).

5. Mecanisme de comunicare, optimizare și validare

5.1 DAO Pattern - Implementarea Client-Side | Comunicarea cu serverul

În arhitectura client-server, aplicația JavaFX (clientul) nu se conectează direct la baza de date! Comunicarea se face exclusiv prin cereri HTTP către *REST API-ul* descris mai sus. Pentru a gestiona această comunicare, am implementat șablonul DAO (Data Access Object) pe partea de client, în pachetul *com.osf.coursemanagement.dao*.

Fiecare entitate (Utilizator/Curs/ Modul) are propriul DAO ce ascunde complexitatea cererilor HTTP. De exemplu, controller-ul ferestrei de utilizatori doar apelează *userDAO.getAllUsers().* Nu trebuie să știe cum se face cererea la server sau cum se parsează JSON-ul. Asta face codul din interfață mult mai curat și mai ușor de vizualizat.

Alte clase precum LiveAttendanceDAO sau UserDAO acționează ca intermediari. Controller-ul grafic apelează metoda *dao.getData(),* iar DAO-ul se ocupă de detaliile tehnice de rețea. Din perspectiva tehnologiilor folosite, avem HttpClient (java.net.http.HttpClient) pentru trimiterea efectivă a cererilor (GET, POST) și ObjectMapper (Jackson) pentru a deserializa răspunsul JSON primit de la server în obiecte Java (List/ Map), necesare prelucrării de date în interiorul aplicației.

Fiecare cerere HTTP construită in DAO aduce cu sine, automat, header-ul de autorizare (*Authorization: Basic ...*) preluat din UserSession →serverul știe mereu cine face cererea.

5.2 Serializarea datelor | Jackson

Pentru a transforma obiectele Java în text JSON (pe care îl trimitem la server) și invers, am folosit biblioteca Jackson. Deși Java are mecanisme interne, Jackson poate gestiona automat cazuri mai complicate, cum ar fi listele de obiecte sau formatele de dată, scutinâd de scrierea manuală a codului de conversie.

5.3 Optimizarea calculelor folosind SQL (native queries)

O provocare a fost calcularea insignelor (Badges) și a scorurilor. Inițial, aș fi putut aduce toate datele în Java și să le aș fi calculat acolo, dar asta ar fi însemnat să transfer multe date inutile prin rețea. Am ales să scriu interogări SQL native (native queries) în repository-urile Spring de pe server și prin acest mijloc baza de date face calculele (sume, medii) și returnează doar rezultatul final, crescând performanța aplicației în termeni de eficiență. Acest lucru este util în special pentru rubricile de "Stats" aferente fiecărei opțiuni / panel din Dashboard. Calculul propriu-zis pentru Superbadges (mai mult de 50% = 3 insigne, iar sub 50% = 1 badge obținut) se face în Java (BadgesController /ModuleService).

5.4 (Warning) dialogs și verificările de condiții

Așa cum se cere, am pus accent pe evitarea greșelilor accidentale:

- Ștergerea **n**u se face niciodată direct. Când se apasă butonul *"Delete"*, apare o fereastră de confirmare. Deci, abia după ce utilizatorul confirmă explicit, se trimite comanda la server.
- Formularele verifică dacă câmpurile obligatorii sunt completate corect (de exemplu, formatul de email) înainte de a permite salvarea

5.5 Dinamica interfeței | Adaptabilitatea UI

Interfața se schimbă în funcție de cine este logat. Folosind informația din UserSession, aplicația ascunde sau afișează butoane. Din backend, rezultatul completării unor câmpuri se reflectă la nivel vizual din mai multe forme. De exemplu, când backend-ul aruncă o eroare (ex: email duplicat), UserDAO prinde eroarea. Interfața (UserManagementController) *preia trigger-ul (impropriu spus)* și afișează mesajul într-un label special (lblWarning) care este stilizat cu roșu.

Un alt exemplu e cel în care un colaborator nu va vedea butonul de "Delete Course", fiind limitat la gestionarea progresului participanților, dar un administrator îl va vedea. În plus, ceilalți membri din staff au acces interzis la aplicație. Această logică este dublată și pe server, unde API-ul respinge cererile neautorizate, pentru securitate.

6 Funcționalitățile aplicației (User experience)

Această secțiune descrie fluxul complet de utilizare și comportamentele specifice ale fiecărei ferestre, evidențiind detaliile de implementare menite să îmbunătățească experiența utilizatorului (UX). Pentru pornirea aplicației, se deschide folderul proiectului

într-un Java IDE sau/și din două terminale se pornește serverul (cu ..*mvnw spring-boot:run*), respectiv clientul (cu .\mvnw clean javafx:run pentru recompilare).

6.1 Autentificarea (Login screen)

Reprezintă primul punct de contact al utilizatorului cu aplicația (și probabil și ultimul, dacă se decide să se deconecteze înainte să închidă programul). Utilizatorul introduce email-ul și parola, iar la apăsarea butonului *Login* aplicația trimite un request POST către server. Sub forma unui feedback vizual, dacă serverul răspunde cu eroare (cod 401), se afișează un mesaj sugestiv (exception) (exemplu: *"Invalid email or password! Try again!" / "Access denied. Only Admins/Staff can use this console."*) direct în interfață, nu doar în consolă.

Parola este mascată automat (PasswordField) pentru securitate. La succes, obiectul User primit este stocat în UserSession (Singleton), iar utilizatorul este redirecționat către Dashboard cu drepturile specifice rolului său. Înainte de asta, fereastra de Authentication Successful cu un *progress bar* ce se schimbă la fiecare secundă își face prezența.

6.2 Centrul de Comandă (Dashboard)

Este ecranul principal (meniul) care se adaptează dinamic. Conține în marginea superioară a BorderPane-ului mesajul "Welcome, [Prenume]" , alături de rolul user-ului, o imagine flexibilă în stânga, ce se modifică în funcție de momentul zilei și de o imagine ce-și schimbă nuanțele la comutarea dark / light mode. Sidebar-ul din stânga conține butoane mari, clare, care se activează în funcție de rol (colaboratorii nu văd "Manage Users", de exemplu). *Stats* aduce elemente de statistică prin ferestre de tip dialog, dispuse on-click.

6.3 Gestiunea utilizatorilor (User management)

Un modul complex dedicat tuturor userilor înscriși în baza de date (admin / participant/ colaborator/ instructor). Este accesibilă și vizibilă (alături de Course și Module doar pentru administrator! Dinperspectiva afișajului, secțiunea utilizează un *ListView* personalizat. Selectarea unui utilizator declanșează un listener care populează instantaneu panoul cu detalii din dreapta. Evident, nu afișează doar părțile statice, ci face calcule. De exemplu, afișează numele Trainer-ului asociat studentului (prin căutare ID în listă) și un tabel cu progresul academic (note, insigne, cursuri la care s-a făcut înscrierea).

Un alt aspect ce se păstrează pentru restul ferestrelor de management îl reprezintă persistența temei. Butonul de "(Light/)Dark Mode" din colț apelează *UserSession.setDarkMode()*, asigurându-se că preferința utilizatorului este reținută, chiar dacă schimbă ferestrele.

6.4 Managementul cursurilor (Hierarchical view)

În mare parte se reciclează din mecanismul UI de la Users. Lista de cursuri permite inspectarea detaliilor, iar listener-ul special (*setOnMouseClicked* cu "*clickCount == 2*") deschide automat fereastra de editare rapidă (dialog) pentru cursul selectat. Detaliile complexe (număr înscrieri, lista cu module) sunt aduse de pe server pe un fir de execuție separat (Task<Map>), astfel încât interfața nu "îngheață" cât timp se așteaptă răspunsul bazei de date, deci putem face referire la un algoritm paralel cu aceast[ocazie.

6.5 Gamification și insigne (Badges)

Este partea de management cea mai interesantă, întrucât este punctea de legătură între tabelele / opțiunile (ce păreau) de sine stătătoare. Utilizatorul alege întâi un Curs (ComboBox), ceea ce populează automat lista de studenți înscriși la acel curs.

În lista de module, cele marcate ca "SuperBadge" sunt evidențiate vizual cu text portocaliu și eticheta [SUPERBADGE]. Logica din spate acordă automat 3 insigne dacă scorul la un astfel de modul depășește 50%, așa cum am explicat și în secțiunea 4.3. Câmpurile de introducere a scorului au un listener care respinge orice caracter care nu este cifră, prevenind erori de introducere a datelor. Punctele preluate de la module sunt scalate de la 0 la 100, iar fiecare modul are aceeași pondere în punctajul (nota finală componentă asincronă).

Secțiunea LiveAttendance afișează sesiunile live programate, iar numele trainer-ului este rezolvat automat (dacă sesiunea conține doar TrainerID, controller-ul face un apel rapid la UserDAO pentru a afișa numele complet al profesorului). Datele calendaristice sunt formatate automat în standardul ISO pentru claritate**.**

6.6 Dark Mode

O funcționalitate modernă adăugată este suportul pentru Dark Mode, iar implementarea nu se bazează, de fapt, pe biblioteci externe, ci pe manipularea adecvată a CSS-ului JavaFX.

În CSS (style.css), marea partea a customizării se rezumă la clasa CSS .dark-theme; am definit și suprascrieri specifice:

.dark-theme { -fx-background-color: #2c3e50; }

.dark-theme .label { -fx-text-fill: white; }

Astfel, când această clasă este adăugată pe nodul rădăcină al ferestrei, toate culorile se inversează automat conform regulilor definite. Preferința utilizatorului este reținută într-o variabilă booleană *isDarkMode* în Singleton-ul *UserSession*. Asta înseamnă că dacă Dark Mode e activat în Dashboard sau în oricare altă fereastră, setarea rămâne activă și când se navighează în alte ferestre. Fiecare Controller are metoda *applyTheme()* apelată la inițializare (initialize()). Aceasta verifică Singleton-ul și adaugă sau scoate clasa dark-theme de pe containerul principal, în funcție de opțiune.

7 Dorințe și perspective de îmbunătățire

Evident, când vine vorba de posibilele rafinări, optimizări suplimentare sau îmbunătățiri ale proiectului, se ivesc concomitent multe idei. Dar în situația de față mai contează și modalitatea de incorporare, abilitățile necesare, răbdarea și cel mai important, ideea de a nu strica din folosul elementelor primordiale doar de dragul de a arăta o grămadă de caracteristici impresionante pe hârtie. Și în acest caz, se mai putea rafina puțin dark mode-ul, care nu e încă optimizat complet pentru partea de progres asincron sau de a introduce data înscrierii în Enrolments (aspect uitat la implementare, field-ul EnrolmentDate fiind prezent în SQL. Aș fi optimizat afișarea tabelelelor privind situația participanților la curs și totodată aș fi extins idea de design /ferestre responsive la diferite rezoluții. Pentru a extinde, în schimb cu adevărat funcționalitatea și a crește valoarea aplicației, aș mai propune următoarele posibilități de îmbunătățire:

- extinderea sistemului de insigne prin introducerea unui clasamnet global și a nivelurilor de experiență. Studenții ar putea primi notificări vizuale ("Daily Streak") pentru participarea consecutivă la cursurile live, crescând astfel engagement-ul.
- integrarea unui serviciu de email (ex: JavaMailSender) pentru a trimite notificări automate: când un student primește o notă nouă, cu o oră înainte de începerea unei sesiuni Live.
- Implementarea unei secțiuni de "Analytics" în Dashboard, folosind librăria JavaFX Charts. Aceasta ar afișa grafice de tip PieChart sau LineChart pentru a vizualiza evoluția notelor unui student pe parcursul anului sau distribuția

notelor pe un curs, pe seama statisticilor și a unor proceduri mai complexe de calcul.-

- Integrarea accesului participanților și trainerilor la aplicație, cu schimbări vizuale și limitare a posibilăților de modificare a datelor (sensibile).

(În paginile următoare sunt illustrate bucăți de cod cu mecanisme integrale din cadrul aplicației)

Mecanismul de preluare și calcul a statisticilor pentru Badges

```java
public class AuthService {
    // statistici user (insigne si scoruri)
    public List<Map<String, Object>> getUserCourseStats(int userId) {
        List<Map<String, Object>> result = java.util.ArrayList.class.cast(new java.util.ArrayList<>());
        List<Course> courses = courseRepository.findCoursesByUserId(userId);

        for (Course c : courses) {
            Map<String, Object> stats = new HashMap<>();
            stats.put("courseId", c.getCourseId());
            stats.put("courseName", c.getName());

            // Badges
            Integer badges = progressRepository.countBadgesForUserAndCourse(userId, c.getCourseId());
            stats.put("badges", badges != null ? badges : 0);

            // progres al score ului prin modulele asincrone (punctajul total com asinc=
            // suma pct trailhead / nr total module din curs)
            Integer totalModules = moduleRepository.countModulesByCourseId(c.getCourseId());
            Double totalProgressPoints = progressRepository.findTotalScoreByUserIdAndCourseId(userId, c.getCourseId());

            if (totalModules == null)
                totalModules = 0;
            if (totalProgressPoints == null)
                totalProgressPoints = 0.0;

            double pScore = 0.0;
            if (totalModules > 0) {
                pScore = totalProgressPoints / totalModules;
            }
            stats.put("progressScore", pScore);

            // Live Score (Real Average: punctajul total com live= suma pct sesiune / nr
            // total sesiuni live din curs
            Integer totalSessions = liveSessionRepository.countSessionsByCourseId(c.getCourseId());
            Double totalLivePoints = liveAttendanceRepository.findTotalScoreByUserIdAndCourseId(userId,
                    c.getCourseId());

            if (totalSessions == null)
                totalSessions = 0;
            if (totalLivePoints == null)
                totalLivePoints = 0.0;

            double lScore = 0.0;
            if (totalSessions > 0) {
                lScore = totalLivePoints / totalSessions;
            }
            stats.put("liveScore", lScore);

            // Total Score
            stats.put("totalScore", (pScore + lScore) / 2.0);

            result.add(stats);
        }
        return result;
    }
```

AuthController (parțial)

```java
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private AuthService authService;

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody Map<String, String> credentials) {
        String email = credentials.get("email");
        String password = credentials.get("password");

        if (email == null || password == null) {
            return ResponseEntity.badRequest().body("Email and password are required");
        }

        try {
            User user = authService.authenticate(email, password);
            return ResponseEntity.ok(user);
        } catch (RuntimeException e) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(e.getMessage());
        }
    }

    // conf register (Register generic)
    @PostMapping("/register")
    public ResponseEntity<?> register(@RequestBody User user) {
        try {
            return ResponseEntity.ok(authService.registerUser(user));
        } catch (RuntimeException e) {
            return ResponseEntity.badRequest().body(e.getMessage());
        }
    }
```

Badges Controller - init + preluarea datelor despre trainees

```java
public class BadgesController {
    private final Map<Integer, Integer> progressIds = new HashMap<>();

    private List<Module> currentModules = new ArrayList<>();
    private TraineeItem selectedTrainee;

    @FXML
    public void initialize() {
        // incarc cursuri
        List<Course> courses = courseDAO.getAllCourses();
        comboCourses.setItems(FXCollections.observableArrayList(courses));

        // listeners care sunt apelate/ "declansate atunci cand sunt selectate cursuri
        // sau studenti
        comboCourses.getSelectionModel().selectedItemProperty().addListener((obs, oldV, newV) -> {
            if (newV != null)
                loadTrainees(newV);
        });

        listTrainees.getSelectionModel().selectedItemProperty().addListener((obs, oldV, newV) -> {
            if (newV != null)
                showDetails(newV);
            else
                hideDetails();
        });

        // tema
        setTheme(UserSession.getInstance().isDarkMode());
    }

    private void loadTrainees(Course course) {
        listTrainees.getItems().clear();
        hideDetails();

        List<Map<String, Object>> participants = courseDAO.getParticipants(course.getId());

        List<TraineeItem> items = participants.stream()
                .map(p -> new TraineeItem(
                        (String) p.get("studentName"),
                        (Integer) p.get("enrolmentId"),
                        (Integer) p.get("userId")))
                .collect(Collectors.toList());

        listTrainees.setItems(FXCollections.observableArrayList(items));
    }
```

Badges Controller - showUserDialog**) -** validarea datelor și interacțiunea cu utilizatorul prin ferestre pop up (ferestre modale, afișare erori); Am folosit Java 8 (Streams) pentru

a filtra lista de utilizatori în memorie, excluzând cei deja înscriși, fără a face request-uri inutile la bază.

```java
@FXML
private void handleAddStudent() {
    Course course = comboCourses.getValue();
    if (course == null) {
        new Alert(Alert.AlertType.WARNING, "Please select a course first.").show();
        return;
    }

    // dialog alegere user
    Dialog<User> dialog = new Dialog<>();
    dialog.setTitle("Add Student");
    dialog.setHeaderText("Enroll a student to " + course.getName());

    ButtonType enrollBtn = new ButtonType("Enroll", ButtonBar.ButtonData.OK_DONE);
    dialog.getDialogPane().getButtonTypes().addAll(enrollBtn, ButtonType.CANCEL);

    GridPane grid = new GridPane();
    grid.setHgap(10);
    grid.setVgap(10);

    ComboBox<User> userCombo = new ComboBox<>();
    userCombo.setPromptText("Select User...");

    // incarc useri care NU sunt deja inscrisi
    // iau toti userii
    List<User> allUsers = userDAO.getAllUsers();
    // iau id-urile participantilor curenti
    // refolosesc courseDAO.getParticipants pt userIds
    List<Integer> currentParticipantIds = courseDAO.getParticipants(course.getId()).stream()
            .map(p -> (Integer) p.get("userId"))
            .collect(Collectors.toList());

    // Filter
    List<User> available = allUsers.stream()
            .filter(u -> !currentParticipantIds.contains(u.getId()))
            .filter(u -> !"Admin".equalsIgnoreCase(u.getRoleType())) // Optional: filter admins?
            .collect(Collectors.toList());

    userCombo.setItems(FXCollections.observableArrayList(available));
    userCombo.setCellFactory(lv -> new ListCell<>() {
        @Override
        protected void updateItem(User item, boolean empty) {
            super.updateItem(item, empty);
            setText(empty ? "" : item.getFirstName() + " " + item.getLastName() + " (" + item.getEmail() + ")");
        }
    });
    userCombo.setButtonCell(userCombo.getCellFactory().call(null)); // Button rendering

    grid.add(new Label("Student:"), 0, 0);
    grid.add(userCombo, 1, 0);
    dialog.getDialogPane().setContent(grid);

    // elem de stil
    if (UserSession.getInstance().isDarkMode()) {
        dialog.getDialogPane().getStylesheets()
                .add(getClass().getResource("/com/osf/coursemanagement/style.css").toExternalForm());
        dialog.getDialogPane().getStyleClass().add("dark-theme");
    }

    dialog.setResultConverter(btn -> {
        if (btn == enrollBtn)
            return userCombo.getValue();
        return null;
    });

    Optional<User> res = dialog.showAndWait();
    if (res.isPresent()) {
        User u = res.get();
        boolean ok = enrolmentDAO.enrol(u.getId(), course.getId());
        if (ok) {
            loadTrainees(course); // refresh
        } else {
            new Alert(Alert.AlertType.ERROR, "Failed to enrol student.").show();
        }
    }
}
```

```java
@Repository
public interface SelfPacedProgressRepository extends JpaRepository<SelfPacedProgress, Integer> {

    // aplicare sql nativ (insert, update, delete) / crud

    @org.springframework.data.jpa.repository.Modifying
    @org.springframework.transaction.annotation.Transactional
    @org.springframework.data.jpa.repository.Query(value = "INSERT INTO SelfPacedProgress (EnrolmentID, ModuleID, CompletionDate, NumberOfPoints, Score) VALUES (:enrolmentId, :mo
    void insertProgressNative(
            @org.springframework.data.repository.query.Param("enrolmentId") int enrolmentId,
            @org.springframework.data.repository.query.Param("moduleId") int moduleId,
            @org.springframework.data.repository.query.Param("completionDate") java.time.LocalDate completionDate,
            @org.springframework.data.repository.query.Param("numberOfPoints") int numberOfPoints,
            @org.springframework.data.repository.query.Param("score") int score);

    @org.springframework.data.jpa.repository.Modifying
    @org.springframework.transaction.annotation.Transactional
    @org.springframework.data.jpa.repository.Query(value = "UPDATE SelfPacedProgress SET CompletionDate = :completionDate, NumberOfPoints = :numberOfPoints, Score = :score WHERE
    void updateProgressNative(
            @org.springframework.data.repository.query.Param("progressId") int progressId,
            @org.springframework.data.repository.query.Param("completionDate") java.time.LocalDate completionDate,
            @org.springframework.data.repository.query.Param("numberOfPoints") int numberOfPoints,
            @org.springframework.data.repository.query.Param("score") int score);

    @org.springframework.data.jpa.repository.Modifying
    @org.springframework.transaction.annotation.Transactional
    @org.springframework.data.jpa.repository.Query(value = "DELETE FROM SelfPacedProgress WHERE ProgressID = :progressId", nativeQuery = true)
    void deleteProgressNative(@org.springframework.data.repository.query.Param("progressId") int progressId);

    @org.springframework.data.jpa.repository.Query(value = "SELECT SUM(sp.Score) FROM SelfPacedProgress sp JOIN Enrolments e ON sp.EnrolmentID = e.EnrolmentID WHERE e.UserID = :u
    Double findTotalScoreByUserIdAndCourseId(@org.springframework.data.repository.query.Param("userId") int userId,
            @org.springframework.data.repository.query.Param("courseId") int courseId);
    Ctrl+I for Command, Ctrl+L for Agent
    java.util.List<SelfPacedProgress> findByEnrolmentId(int enrolmentId);

    java.util.List<SelfPacedProgress> findByModuleId(int moduleId);

    // nr badges pt user - pt superbadge e 3 la un scor >= 50%,
    // respectiv 1 badge pt < 50 %, pt badge e 1 indif de punctaj
    @org.springframework.data.jpa.repository.Query(value = "SELECT SUM(CASE WHEN TRIM(UPPER(m.ModuleType)) = 'SUPERBADGE' THEN (CASE WHEN sp.Score >= 50 THEN 3 WHEN sp.Score > 0
    Integer countBadgesForUser(@org.springframework.data.repository.query.Param("userId") int userId);

    @org.springframework.data.jpa.repository.Query(value = "SELECT SUM(CASE WHEN TRIM(UPPER(m.ModuleType)) = 'SUPERBADGE' THEN (CASE WHEN sp.Score >= 50 THEN 3 WHEN sp.Score > 0
    Integer countBadgesForUserAndCourse(@org.springframework.data.repository.query.Param("userId") int userId,
            @org.springframework.data.repository.query.Param("courseId") int courseId);
}
```

↑ Mecanism Native Query (in loc de jpa- save()/ delete()) pentru progresul asincron

↓ Mecanism Native Query (SQL) pentru Module

```java
@Repository
public interface ModuleRepository extends JpaRepository<Module, Integer> {
    // gaseste toate modulele unui curs
    List<Module> findByCourseId(int courseId);

    @org.springframework.data.jpa.repository.Query(value = "SELECT COUNT(*) FROM Modules_Trailhead WHERE CourseID = :courseId", nativeQuery = true)
    Integer countModulesByCourseId(@org.springframework.data.repository.query.Param("courseId") int courseId);

    // aplicare sql nativ (insert, update, delete) / crud - SQL nativ
    @org.springframework.data.jpa.repository.Modifying
    @org.springframework.transaction.annotation.Transactional
    @org.springframework.data.jpa.repository.Query(value = "INSERT INTO Modules_Trailhead (CourseID, Name, Description, CompletionCriteria, ModuleType, NumberOfPoints) "
            +
            "VALUES (:courseId, :name, :description, :completionCriteria, :moduleType, :numberOfPoints)", nativeQuery = true)
    void insertModuleNative(
            @org.springframework.data.repository.query.Param("courseId") int courseId,
            @org.springframework.data.repository.query.Param("name") String name,
            @org.springframework.data.repository.query.Param("description") String description,
            @org.springframework.data.repository.query.Param("completionCriteria") String completionCriteria,
            @org.springframework.data.repository.query.Param("moduleType") String moduleType,
            @org.springframework.data.repository.query.Param("numberOfPoints") int numberOfPoints);

    @org.springframework.data.jpa.repository.Modifying
    @org.springframework.transaction.annotation.Transactional
    @org.springframework.data.jpa.repository.Query(value = "UPDATE Modules_Trailhead SET CourseID = :courseId, Name = :name, Description = :description, "
            +
            "CompletionCriteria = :completionCriteria, ModuleType = :moduleType, NumberOfPoints = :numberOfPoints WHERE ModuleID = :moduleId", nativeQuery = true)
    void updateModuleNative(
            @org.springframework.data.repository.query.Param("moduleId") int moduleId,
            @org.springframework.data.repository.query.Param("courseId") int courseId,
            @org.springframework.data.repository.query.Param("name") String name,
            @org.springframework.data.repository.query.Param("description") String description,
            @org.springframework.data.repository.query.Param("completionCriteria") String completionCriteria,
            @org.springframework.data.repository.query.Param("moduleType") String moduleType,
            @org.springframework.data.repository.query.Param("numberOfPoints") int numberOfPoints);

    @org.springframework.data.jpa.repository.Modifying
    @org.springframework.transaction.annotation.Transactional
    @org.springframework.data.jpa.repository.Query(value = "DELETE FROM Modules_Trailhead WHERE ModuleID = :moduleId", nativeQuery = true)
    void deleteModuleNative(@org.springframework.data.repository.query.Param("moduleId") int moduleId);
}
```